

Descripción del Diagrama de Arquitectura

Embedded Payment Platform – Diagrama de paquetes y componentes

Juan Esteban Mosquera Perea

Maria Paula Mosquera Alvarez

Sebastian Suarez Restrepo

Enlace Diagrama

El diagrama presentado describe la arquitectura lógica de una **plataforma de pagos embebida** que permite a comercios integrar capacidades de procesamiento de pagos dentro de sus aplicaciones o sistemas. Esta plataforma actúa como intermediario entre los clientes que realizan pagos y los sistemas del comercio, proporcionando funcionalidades como gestión de pagos, registro de transacciones, reembolsos, autenticación y control de acceso.

La arquitectura del sistema se basa en el patrón de **Monolito Modular por Capas**, el cual organiza el sistema en módulos claramente delimitados que mantienen responsabilidades separadas y promueven la mantenibilidad, escalabilidad y claridad estructural del software.

El sistema está organizado en seis grandes grupos de módulos:

- Client Applications
- Presentation Layer
- Application Layer
- Domain Layer
- Cross-Cutting Modules
- Infrastructure Layer

Cada uno de estos grupos representa un nivel diferente de responsabilidad dentro del sistema.

1. Client Applications (Componentes no implementados)

El diagrama incluye una capa de **aplicaciones cliente**, que representa las interfaces desde las cuales los usuarios o comercios interactuarían con la plataforma de pagos.

Los componentes incluidos son:

- **Web Frontend**
- **Mobile App**

Estas aplicaciones representarían, en un sistema completo, las interfaces de usuario que permitirían a los comerciantes administrar sus pagos, consultar transacciones o gestionar configuraciones de la plataforma.

Sin embargo, **estos componentes no serán implementados en el alcance de este proyecto**, ya que el desarrollo se enfocará exclusivamente en el **backend de la plataforma de pagos**.

Por esta razón, estos elementos se encuentran marcados en el diagrama con un color diferente y el estereotipo <>, indicando que forman parte de la arquitectura conceptual pero no del prototipo desarrollado.

Estas aplicaciones cliente interactuarían con la plataforma a través de las **APIs expuestas en la capa de presentación**.

2. Presentation Layer (API)

La **Presentation Layer** representa el punto de entrada al sistema. En esta capa se exponen las funcionalidades del

backend mediante **controladores de API**, que reciben las solicitudes HTTP provenientes de las aplicaciones cliente o de integraciones externas.

Los controladores se organizan por dominio funcional en diferentes paquetes.

2.1 Auth API

Gestiona los procesos de autenticación y control de sesiones de usuario.

Componentes principales:

- **AuthController**: maneja las operaciones de inicio de sesión y registro de usuarios.
- **MFAController**: gestiona los procesos de autenticación multifactor (MFA).

Estos controladores invocan los casos de uso del módulo de autenticación en la capa de aplicación.

2.2 Merchant API

Gestiona las operaciones relacionadas con los comercios que utilizan la plataforma.

Componente principal:

- **MerchantController**

Permite realizar operaciones como:

- Registro de comercios
- Consulta de perfiles de comercio
- Generación de claves API para integración

2.3 Payments API

Gestiona las operaciones relacionadas con el procesamiento de pagos.

Componente principal:

- **PaymentController**

Permite a los comercios realizar acciones como:

- Crear una intención de pago
- Confirmar un pago
- Cancelar una operación de pago

2.4 Transactions API

Gestiona la consulta y registro de transacciones financieras.

Componente principal:

- **TransactionController**

Permite obtener el historial de transacciones de un comercio.

2.5 Refunds API

Gestiona los procesos de devolución de pagos.

Componente principal:

- **RefundController**

Permite solicitar el reembolso de transacciones previamente ejecutadas.

2.6 Webhooks API

Permite recibir notificaciones externas sobre eventos del sistema.

Componente principal:

- **WebhookController**

Este componente permite manejar eventos externos como:

- confirmaciones de pago
- cambios de estado de transacciones
- eventos provenientes de integraciones externas

3. Application Layer (Use Cases)

La **Application Layer** contiene la implementación de los **casos de uso** del

sistema. Esta capa orquesta las operaciones necesarias para ejecutar los procesos del negocio utilizando los servicios del dominio y los repositorios.

Cada módulo representa un área funcional del sistema.

3.1 Auth Module

Gestiona los procesos de autenticación y seguridad de usuarios.

Casos de uso incluidos:

- **LoginUseCase**
- **RegisterUserUseCase**
- **VerifyMFAUseCase**
- **RevokeTokenUseCase**

Estos casos de uso interactúan con servicios de seguridad y repositorios de usuarios para validar credenciales y gestionar sesiones.

3.2 Merchant Module

Gestiona el ciclo de vida de los comercios dentro de la plataforma.

Casos de uso principales:

- **RegisterMerchantUseCase**
- **GetMerchantProfileUseCase**
- **GenerateApiKeysUseCase**

3.3 Payments Module

Gestiona la creación y ejecución de pagos.

Casos de uso incluidos:

- **CreatePaymentIntentUseCase**
- **ConfirmPaymentUseCase**
- **CancelPaymentUseCase**

Estos casos de uso coordinan la lógica necesaria para ejecutar pagos y registrar su estado en el sistema.

3.4 Transactions Module

Gestiona el registro y consulta de las transacciones realizadas.

Casos de uso incluidos:

- **RegisterTransactionUseCase**
- **GetTransactionHistoryUseCase**

3.5 Refunds Module

Gestiona las solicitudes de reembolso de pagos.

Caso de uso principal:

- **CreateRefundUseCase**

4. Domain Layer (Business Logic)

La **Domain Layer** representa el núcleo del sistema y contiene las reglas de negocio principales.

Esta capa se divide en tres categorías.

4.1 Entities

Las entidades representan los objetos fundamentales del dominio.

Entre ellas se encuentran:

- **Merchant:** representa a un comercio registrado en la plataforma.
- **Customer:** representa al cliente que realiza un pago.
- **PaymentIntent:** representa una intención de pago previa a su confirmación.
- **Transaction:** representa una operación financiera ejecutada.
- **Refund:** representa una devolución de un pago.
- **Role y Permission:** entidades utilizadas para el control de acceso basado en roles.

4.2 Domain Services

Los servicios de dominio contienen lógica de negocio que involucra múltiples entidades.

Componentes principales:

- **PaymentService**
- **FraudDetectionService**
- **AuthorizationService**

El **PaymentService** coordina el procesamiento de pagos dentro del sistema.

El **AuthorizationService** gestiona la lógica de autorización basada en roles (RBAC).

El **FraudDetectionService** representa un componente destinado a analizar posibles fraudes en transacciones. Sin embargo, **este servicio no será implementado en el alcance del proyecto** y se incluye únicamente para reflejar una arquitectura completa similar a la utilizada en plataformas reales de pago.

Por esta razón, este componente se encuentra marcado como <> en el diagrama.

4.3 Repository Interfaces

Las interfaces de repositorio definen contratos para el acceso a los datos del sistema.

Repositorios incluidos:

- MerchantRepository
- PaymentRepository
- TransactionRepository
- RefundRepository
- CustomerRepository
- RoleRepository

Estas interfaces permiten desacoplar la lógica del dominio de las tecnologías específicas de persistencia.

5. Cross-Cutting Modules

Los **Cross-Cutting Modules** contienen funcionalidades transversales que son utilizadas por múltiples módulos del sistema.

5.1 Security

Proporciona mecanismos de seguridad para el sistema.

Componentes principales:

- **JWTService**
- **PasswordPolicyService**
- **TokenBlacklistService**
- **ApiKeyValidator**

Estos servicios permiten gestionar autenticación basada en tokens, validación de contraseñas y control de sesiones.

5.2 RBAC Authorization

Gestiona el control de acceso basado en roles.

Componente principal:

- **PermissionChecker**

Este módulo valida que los usuarios tengan los permisos necesarios para acceder a ciertos recursos o ejecutar determinadas operaciones.

5.3 Audit

Permite registrar eventos relevantes del sistema para auditoría.

Componentes:

- **AuditService**
- **AuditEvent**

Estos registros permiten rastrear acciones críticas como accesos al sistema o cambios en el estado de transacciones.

5.4 Logging & Observability

Facilita el monitoreo y registro de eventos del sistema.

Componentes:

- **Logger**
- **TraceIdGenerator**

Estos elementos permiten generar registros estructurados que facilitan la observabilidad del sistema.

5.5 Error Handling

Centraliza el manejo de errores del sistema.

Componentes:

- **GlobalErrorHandler**
- **ApiErrorResponse**

Este módulo garantiza que los errores se devuelvan en un formato consistente a los consumidores de la API.

6. Infrastructure Layer

La **Infrastructure Layer** contiene las implementaciones concretas necesarias para interactuar con sistemas externos y recursos técnicos.

6.1 Persistence

Implementa el acceso a la base de datos mediante repositorios concretos que implementan las interfaces definidas en el dominio.

Componentes principales:

- **DatabaseConnection**
- **MerchantRepositoryImpl**
- **PaymentRepositoryImpl**

- **TransactionRepositoryImpl**
- **RefundRepositoryImpl**
- **CustomerRepositoryImpl**
- **RoleRepositoryImpl**

6.2 Cache

Proporciona mecanismos de almacenamiento temporal para mejorar el rendimiento del sistema.

Componente principal:

- **RedisCache**

6.3 API Documentation

Gestiona la generación automática de documentación para las APIs.

Componente:

- **SwaggerConfig**

6.4 Webhooks Delivery

Permite enviar notificaciones a sistemas externos cuando ocurren eventos relevantes.

Componente:

- **WebhookDispatcher**

7. Componentes fuera del alcance del proyecto

Algunos componentes incluidos en el diagrama forman parte de una **arquitectura conceptual completa**, pero no serán implementados en el desarrollo del prototipo.

Estos componentes son:

- **Web Frontend**
- **Mobile App**
- **FraudDetectionService**

La inclusión de estos elementos en el diagrama permite representar cómo se integraría el sistema en un entorno real de producción, aunque su desarrollo no forma parte del alcance actual del proyecto.